Failure scenarios Control Plane vs Worker Node

==========

Nice choice 😄 — this topic **separates real-world engineers from theory readers**.
Below is a **clear, interview-ready breakdown of failure scenarios**, exactly how interviewers expect you to explain.

Kubernetes Failure Scenarios

Control Plane vs Worker Node

1️⃣ Control Plane Failure Scenarios

1. API Server Failure

What happens:

- kubectl stops working
- No new Pods can be created or updated
- Existing Pods keep running

Why:
API Server is the single entry point for all cluster operations.

Interview line:

If API Server is down, the cluster becomes read-only but workloads continue running.

---

2. etcd Failure

What happens:

- Cluster state is lost or inconsistent
- Scheduler and controllers cannot function
- Cluster recovery becomes difficult

Why:
etcd stores the entire desired and current state of Kubernetes.

Interview line:

etcd failure is the most critical failure in Kubernetes.

---

3. Scheduler Failure

What happens:

- New Pods stay in Pending state
- Existing Pods are unaffected

Why:
Scheduler is responsible only for placing new Pods.

Interview line:

Scheduler failure affects only new workload placement, not running Pods.

---

4. Controller Manager Failure

What happens:

- Auto-healing stops
- Replicas are not recreated
- Node failures are not handled

Why:
Controllers reconcile desired vs actual state.

Interview line:

Without controllers, Kubernetes loses its self-healing capability.

---

5. Control Plane Node Failure

What happens:

- Entire cluster becomes unmanaged
- No scaling, no updates
- Workloads continue until something breaks

Why:
Control Plane nodes host all decision-making components.

Interview line:

Control Plane failure turns Kubernetes into a static container platform.

---

2 Worker Node Failure Scenarios

---

6. Worker Node Crash (VM/Hardware Failure)

What happens:

- Pods on that node go down
- Node marked NotReady
- Pods rescheduled to other nodes

Why:
Kubernetes detects node heartbeat failure via kubelet.

Interview line:

Kubernetes reschedules Pods when a worker node fails.

---

7. kubelet Failure

What happens:

- Pods stop being managed
- Node becomes NotReady
- No new Pods scheduled on that node

Why:
kubelet is responsible for Pod lifecycle on the node.

Interview line:

kubelet failure isolates the node from the cluster.

---

8. Container Runtime Failure

What happens:

- Containers cannot start or restart
- Pod creation fails
- Existing containers may crash

Why:
Runtime executes containers on behalf of kubelet.

Interview line:

Without a container runtime, Pods cannot run.

---

9. kube-proxy Failure

What happens:

- Service networking breaks
- Pod-to-Service communication fails
- Pod-to-Pod may still work

Why:
kube-proxy manages Service routing rules.

Interview line:

kube-proxy failure affects Services, not Pod creation.

🔟 **Resource Exhaustion (CPU/Memory/Disk)**

What happens:

- Pods get evicted
- Node goes into pressure states
- Scheduling avoids the node

Why:
Kubelet enforces resource limits and eviction policies.

Interview line:

Resource pressure is the most common worker node failure in production.

3️⃣ Control Plane vs Worker Node – Failure Comparison Table

Aspect	Control Plane Failure	Worker Node Failure
Scope of Impact	Cluster-wide	Node-specific
Running Pods	Continue running	Pods on node fail
Scheduling	Stops	Continues
Auto-healing	Disabled	Enabled
API Access	Unavailable	Available
Severity	Very High	Medium
Recovery Complexity	High	Low
HA Requirement	Mandatory	Recommended

4️⃣ Golden Interview Summary (Memorize This 🔥)

Control Plane failure stops decisions but workloads continue.
Worker Node failure stops workloads but decisions continue.

5️⃣ Real Interview Scenario Question (Bonus)

Q: If a worker node goes down but control plane is healthy, what happens?
A: Kubernetes detects the failure, marks the node NotReady, and reschedules Pods to healthy nodes.

Q: If control plane goes down but worker nodes are healthy, what happens?
A: Existing Pods keep running, but no new changes or healing occur.

Alright 😎 now we're in **senior / lead / architect interview territory**.
I'll split this cleanly into **two sections**:

1️⃣ Step-by-step troubleshooting for each failure
2️⃣ Ultra-hard real production failure interview questions (with expected thinking)

1️⃣ Troubleshooting Steps for Kubernetes Failures

(Interview-grade, command-driven, production-oriented)

A. API Server Failure

Symptoms

- kubectl hangs or times out
- New deployments fail
- Existing Pods keep running

Troubleshooting Steps

1. Check API server Pod or service status
2. kubectl get pods -n kube-system
3. Verify API server process on control plane
4. systemctl status kube-apiserver
5. Check logs
6. journalctl -u kube-apiserver
7. Validate certificates expiry
8. kubeadm certs check-expiration
9. Restart API server
10. systemctl restart kube-apiserver

Interview insight:

API server issues are often caused by certificate expiry or etcd latency.

B. etcd Failure (Most Critical)

Symptoms

- API Server up but cluster behaves unpredictably
- Pod updates fail
- Controllers stop reconciling

Troubleshooting Steps

1. Check etcd Pod / service
2. kubectl get pods -n kube-system | grep etcd
3. Verify etcd health
4. ETCDCTL_API=3 etcdctl endpoint health
5. Check disk latency and space
6. df -h
7. iostat
8. Inspect logs
9. journalctl -u etcd
10. Restore from snapshot if corrupted
11. etcdctl snapshot restore

Interview insight:

Always keep automated etcd backups — recovery without backup is painful.

C. Scheduler Failure

Symptoms

- Pods stuck in Pending
- No scheduling errors in application logs

Troubleshooting Steps

1. Check scheduler status
2. kubectl get pods -n kube-system | grep scheduler

3. Describe pending Pod
4. kubectl describe pod <pod>
5. Check node resources
6. kubectl describe nodes
7. Validate taints & affinity rules
8. Restart scheduler if needed

Interview insight:

Scheduler failure affects only new Pods, never running ones.

D. Controller Manager Failure

Symptoms

- Replicas not recreated
- Node failures ignored
- HPA not working

Troubleshooting Steps

1. Check controller manager Pod
2. kubectl get pods -n kube-system | grep controller
3. Inspect logs
4. journalctl -u kube-controller-manager
5. Verify leader election
6. Restart controller manager
7. Validate RBAC permissions

Interview insight:

Without controllers, Kubernetes loses self-healing.

E. Worker Node Failure

Symptoms

- Node shows NotReady
- Pods unreachable

Troubleshooting Steps

1. Check node status
2. kubectl get nodes
3. SSH into node
4. Check kubelet
5. systemctl status kubelet
6. Check disk, memory, CPU
7. df -h
8. free -m
9. top
10. Restart kubelet or reboot node

Interview insight:

Most worker node failures are resource-related.

F. kubelet Failure

Symptoms

- Node becomes NotReady
- Pods stuck in ContainerCreating

Troubleshooting Steps

1. Check kubelet logs
2. journalctl -u kubelet
3. Verify API Server connectivity
4. Check certificates

5. Restart kubelet
6. Rejoin node if required
7. kubeadm reset && kubeadm join

G. Container Runtime Failure

Symptoms

- Image pull errors
- Containers crash loop

Troubleshooting Steps

1. Check runtime service
2. systemctl status containerd
3. Check runtime logs
4. Validate disk space
5. Pull image manually
6. Restart runtime

H. kube-proxy Failure

Symptoms

- Service unreachable
- Pod IP works but Service IP fails

Troubleshooting Steps

1. Check kube-proxy Pod
2. Inspect iptables/IPVS
3. Validate CNI plugin
4. Restart kube-proxy
5. Flush iptables if corrupted

2️⃣ Ultra-Hard Real Production Interview Questions 🔥

*(These are **principal engineer level** questions)*

Q1️⃣ Pods are running, but Services suddenly stopped working cluster-wide. What do you check?

Expected thinking:

- kube-proxy failure
- iptables corruption
- CNI plugin crash
- Node kernel update impact

Q2️⃣ API Server is up, but kubectl create pod hangs. Why?

Expected thinking:

- etcd latency or disk pressure
- Admission controller deadlock
- Network issue between API Server and etcd

Q3️⃣ Node shows Ready, but Pods cannot start on it. Why?

Expected thinking:

- Container runtime down
- Disk pressure
- Image filesystem full
- kubelet runtime endpoint mismatch

Q4️⃣ Cluster works fine, but scaling deployments does nothing. Why?

Expected thinking:

- Scheduler down
- Controller manager down
- HPA metrics unavailable

Q5 Pods are restarting even though node is healthy. Why?

Expected thinking:

- Liveness probe misconfiguration
- OOMKilled events
- CPU throttling
- Kernel issues

Q6 After upgrading Kubernetes, Services broke but Pods are healthy. Why?

Expected thinking:

- kube-proxy version mismatch
- iptables vs IPVS mode change
- CNI incompatibility

Q7 etcd is healthy, but cluster state is inconsistent. Why?

Expected thinking:

- Clock drift between nodes
- Split-brain scenario
- Network partition

Q8 Control Plane is HA, but still becomes unavailable sometimes. Why?

Expected thinking:

- Load balancer misconfiguration
- Leader election issues
- Certificate rotation failures

Q9️⃣ Pods are evicted even when node has free memory. Why?

Expected thinking:

- Disk pressure
- PID pressure
- Node allocatable misconfiguration

Q🔟 How do you debug a Kubernetes issue when kubectl itself is down?

Expected thinking:

- SSH to control plane
- Use static Pod logs
- Inspect etcd directly
- Check kubelet logs

Golden Interview Line 🏆

“Most Kubernetes failures are not Kubernetes problems, but Linux, networking, or storage problems.”

Nice 😄 — let’s do this **exactly like a real senior DevOps interview**.
I’ll act as the **interviewer**, you’re the **candidate (10–12+ years exp)**.
I’ll show:

- **Interviewer question**
- ❌ **Weak / junior answer**
- ✅ **Strong / senior answer**
- **What the interviewer is really checking**

No fluff. No theory dumps. Real-world thinking only.

🎯 Mock Senior DevOps Interview (Kubernetes-Focused)

Question 1: Kubernetes Cluster Down, Pods Still Running

Interviewer:

Kubernetes API is down, but applications are still serving traffic. What do you do?

❌ Weak Answer

I will restart the cluster or redeploy the pods.

✅ Strong Senior Answer

Since Pods are running, I know worker nodes and kubelets are healthy.
I would immediately check the API Server and etcd health on the control plane.
My focus would be on certificate expiry, etcd disk latency, or load balancer issues.
I would avoid touching worker nodes to prevent downtime.
Restore control plane functionality without disrupting running workloads.

Interviewer Checks:

- Understanding of control plane vs worker node
- Calm incident response
- Avoiding unnecessary restarts

Question 2: Pods in Pending State for 30 Minutes

Interviewer:

Pods are stuck in Pending state. What's your debugging approach?

❌ Weak Answer

I will check logs and restart pods.

✅ Strong Senior Answer

I would first describe the Pod to check scheduling events.
Then I'd verify scheduler health and node resource availability.
I would check taints, tolerations, and affinity rules.
If resources are insufficient, I'd check node autoscaling.
Restarting Pods is never my first step.

Interviewer Checks:

- Scheduler knowledge
- Structured troubleshooting
- No random actions

Question 3: Services Not Working but Pods Are Reachable

Interviewer:

Pods can talk to each other, but Services stopped working. What happened?

❌ **Weak Answer**

Maybe network issue.

✅ **Strong Senior Answer**

This strongly points to kube-proxy or iptables issues.
I would check kube-proxy Pods and verify iptables/IPVS rules.
Pod networking works, so CNI is likely healthy.
A recent node upgrade or kernel change could have corrupted rules.
Restarting kube-proxy usually fixes this.

Interviewer Checks:

- Deep networking understanding
- Ability to isolate failure domain

Question 4: Worker Node Ready but Pods Fail to Start

Interviewer:

Node shows Ready, but Pods stay in ContainerCreating.

❌ **Weak Answer**

I will restart the node.

✅ **Strong Senior Answer**

Ready only means kubelet heartbeat is working.
I'd check the container runtime status and disk space.
Image pull errors or filesystem full are common causes.
I'd inspect kubelet logs for runtime socket issues.
Reboot is my last option.

Interviewer Checks:

- Difference between Node Ready and workload readiness
- Linux fundamentals

Question 5: Frequent Pod Restarts Without Node Failure

Interviewer:

Pods are restarting even though the node is healthy.

❌ **Weak Answer**

Application issue.

✅ **Strong Senior Answer**

I'd first check Pod events for OOMKilled or probe failures.
Liveness probes are a common misconfiguration.
CPU throttling can also cause false probe failures.
I'd validate resource limits and probe thresholds.
Node health alone doesn't guarantee Pod stability.

Interviewer Checks:

- Resource management knowledge
- Probe understanding

Question 6: etcd Healthy but Cluster State Inconsistent

Interviewer:

etcd shows healthy, but Kubernetes behaves inconsistently.

❌ **Weak Answer**

I will restart etcd.

✅ **Strong Senior Answer**

etcd health doesn't guarantee consistency.
I'd check network latency and clock synchronization.
Split-brain or partial quorum issues are possible.
I'd verify leader election and peer communication.
Restarting etcd blindly can worsen the situation.

Interviewer Checks:

- Distributed systems knowledge
- Risk awareness

Question 7: HPA Not Scaling Pods

Interviewer:

HPA is configured but pods are not scaling.

❌ **Weak Answer**

HPA is not working.

✅ **Strong Senior Answer**

I'd verify Metrics Server availability first.
Then check CPU requests, since HPA uses requests, not limits.
I'd validate HPA events and thresholds.
I'd also check if pods are hitting CPU throttling.
HPA issues are usually configuration or metrics-related.

Interviewer Checks:

- Autoscaling internals
- Metrics understanding

Question 8: Control Plane HA but Still Downtime

Interviewer:

Control plane is HA, but you still see downtime. Why?

❌ **Weak Answer**

Maybe cluster bug.

✅ **Strong Senior Answer**

HA doesn't help if the load balancer is misconfigured.
I'd check health checks and failover timing.
Certificate rotation issues can also cause downtime.
Leader election delays are another possibility.
HA must be validated end-to-end.

Interviewer Checks:

- Real HA experience
- Cloud + Kubernetes integration

Question 9: Debugging When kubectl Is Down

Interviewer:

kubectl is completely down. How do you debug?

❌ **Weak Answer**

I cannot debug.

✅ **Strong Senior Answer**

I'd SSH into control plane nodes.
Check static Pod manifests and kubelet logs.
Inspect etcd health directly.
Use container runtime tools if needed.
kubectl is optional for deep debugging.

Interviewer Checks:

- Confidence under pressure
- OS-level skills

Question 10: One Line – Biggest Production Lesson

Interviewer:

What's your biggest Kubernetes production lesson?

✅ **Senior Answer**

Kubernetes failures usually expose weaknesses in monitoring, capacity planning, or Linux fundamentals—not Kubernetes itself.

Interviewer Reaction:
😎 *Strong hire*

🏆 **How You Win This Interview**

- Never panic
- Never say "restart everything"
- Always isolate failure domain
- Show Linux + networking depth
- Speak calmly and confidently